

Лабораторная работа

Методы оптимизации в нейросетевых вычислениях

Вариант 12

Выполнено: вариант 12

26 мая 2026 г.

Инструменты

Python, NumPy, pandas, Matplotlib, PyTorch, scikit-learn.

Содержание

1	Постановка задачи	3
2	Задание 1. Аналитическое исследование функции	3
2.1	Производные и стационарная точка	3
2.2	Исследование на экстремум	3
2.3	Глобальный минимум на заданной области	4
2.4	Вывод по задаче	4
2.5	Линии уровня	4
3	Задание 2. Градиентный спуск и эвристика Adam	4
3.1	Параметры для застревания вблизи седла	4
3.2	Визуализация траектории GD	5
3.3	Собственная реализация Adam	5
3.4	Сравнение двух наборов гиперпараметров Adam	6
3.5	Готовая реализация Adam из PyTorch	6
3.6	Вывод по заданию 2	7
4	Задание 3. Нейросеть на табличных данных	7
4.1	Выбор датасета	7
4.2	Архитектура модели	8
4.3	Сравнение оптимизаторов	8
4.3.1	Результаты	8
4.3.2	Матрицы ошибок	9
4.3.3	Кривые обучения	9
4.4	Вывод по заданию 3	9
5	Итоговый вывод	9

1 Постановка задачи

Вариант 12 задаёт функцию

$$f(x, y) = 10^{-2}(8x^2 + 2xy - 5x - 2y - 3), \quad (x, y) \in [-20, 20] \times [-50, 50]. \quad (1)$$

Требуется:

1. исследовать функцию аналитически;
2. построить линии уровня;
3. подобрать параметры градиентного спуска, при которых траектория застревает в окрестности седловой точки;
4. реализовать эвристику по варианту — **Adam**;
5. сравнить собственную реализацию с готовой реализацией PyTorch;
6. на табличных данных обучить нейросеть и сравнить стандартный Adam с собственной реализацией.

2 Задание 1. Аналитическое исследование функции

2.1 Производные и стационарная точка

Градиент функции:

$$\nabla f(x, y) = 10^{-2}(16x + 2y - 5, 2x - 2). \quad (2)$$

Стационарная точка находится из системы:

$$16x + 2y - 5 = 0, \quad (3)$$

$$2x - 2 = 0. \quad (4)$$

Отсюда:

$$x^* = 1, \quad (5)$$

$$y^* = -5.5. \quad (6)$$

Значение функции в этой точке:

$$f(1, -5.5) = 0. \quad (7)$$

2.2 Исследование на экстремум

Матрица Гессе постоянна:

$$H = \begin{pmatrix} 0.16 & 0.02 \\ 0.02 & 0 \end{pmatrix}. \quad (8)$$

Её собственные значения:

$$\lambda_1 \approx 0.162462, \quad (9)$$

$$\lambda_2 \approx -0.002462. \quad (10)$$

Так как одно собственное значение положительно, а другое отрицательно, стационарная точка является **седловой**.

Следовательно:

- локального минимума нет;
- локального максимума нет;
- есть одна седловая точка $(1, -5.5)$.

2.3 Глобальный минимум на заданной области

На прямоугольнике $[-20, 20] \times [-50, 50]$ глобальный минимум достигается на границе. Получено:

$$(x_{\min}, y_{\min}) = (-5.9375, 50), \quad (11)$$

$$f(x_{\min}, y_{\min}) = -3.8503125. \quad (12)$$

2.4 Вывод по задаче

Функция обладает седловой точкой и не имеет локальных экстремумов, поэтому для градиентных методов есть риск замедления и «застревания» вблизи седла. На ограниченной области глобальный минимум расположен на границе, то есть обычный спуск должен либо уйти к границе окна, либо продемонстрировать неустойчивую динамику около седла.

2.5 Линии уровня

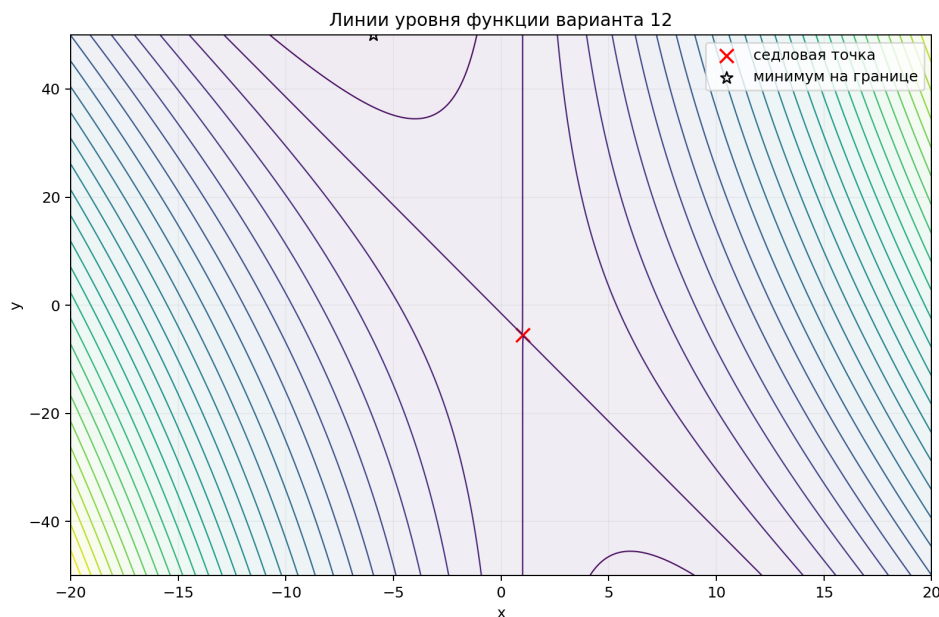


Рис. 1: Линии уровня функции варианта 12

3 Задание 2. Градиентный спуск и эвристика Adam

3.1 Параметры для застревания вблизи седла

Для демонстрации схождения к седловой точке использовались:

- начальное приближение: $x_0 \approx (10.9251, -4.2782)$ — точка на устойчивом направлении относительно седла;

- шаг градиентного спуска: $\alpha = 0.8$;
- критерий останова: $\|\nabla f(x_k)\| < 10^{-6}$.

Результат:

- число итераций: **103**;
- финальная точка: $(1.0000059, -5.4999993)$;
- норма градиента: $9.61 \cdot 10^{-7}$.

3.2 Визуализация траектории GD

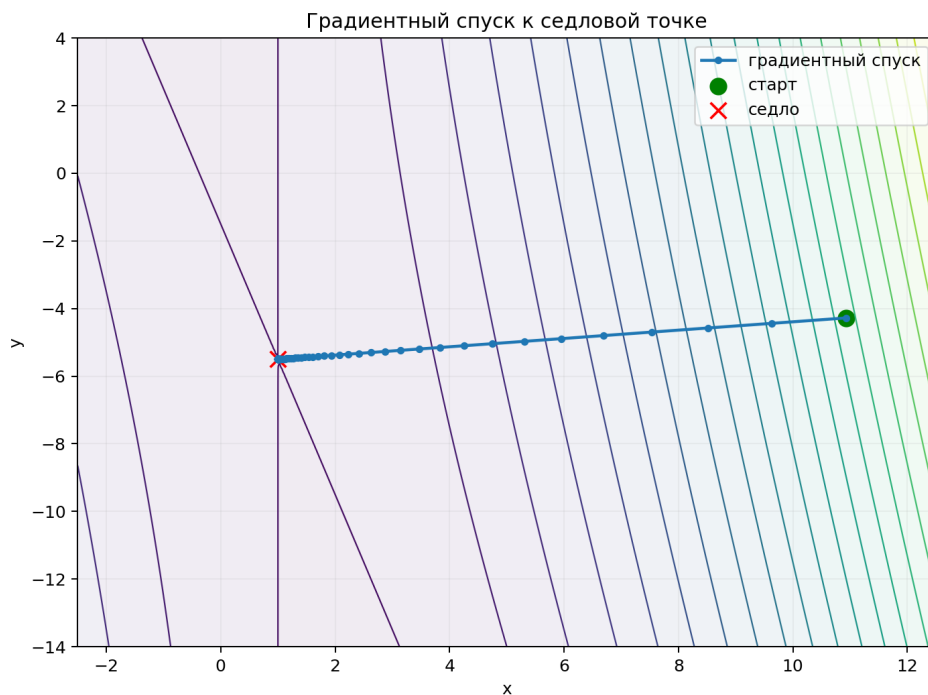


Рис. 2: Градиентный спуск к седловой точке

3.3 Собственная реализация Adam

Для варианта 12 реализован собственный класс оптимизатора `AdamVariant`, совместимый по интерфейсу с `torch.optim.Optimizer`.

Параметры, используемые в демонстрации:

- $lr = 0.1$ — более плавная траектория;
- $lr = 0.2$ — более выраженный «зигзаг»;
- $\text{betas} = (0.9, 0.999)$;
- число итераций: **104** (то же число, что и выше).

3.4 Сравнение двух наборов гиперпараметров Adam

На одной и той же области отображения получаются две характерные траектории:

1. **более плавный ход** — при меньшем шаге;
2. **пилообразная ломаная** — при большем шаге.

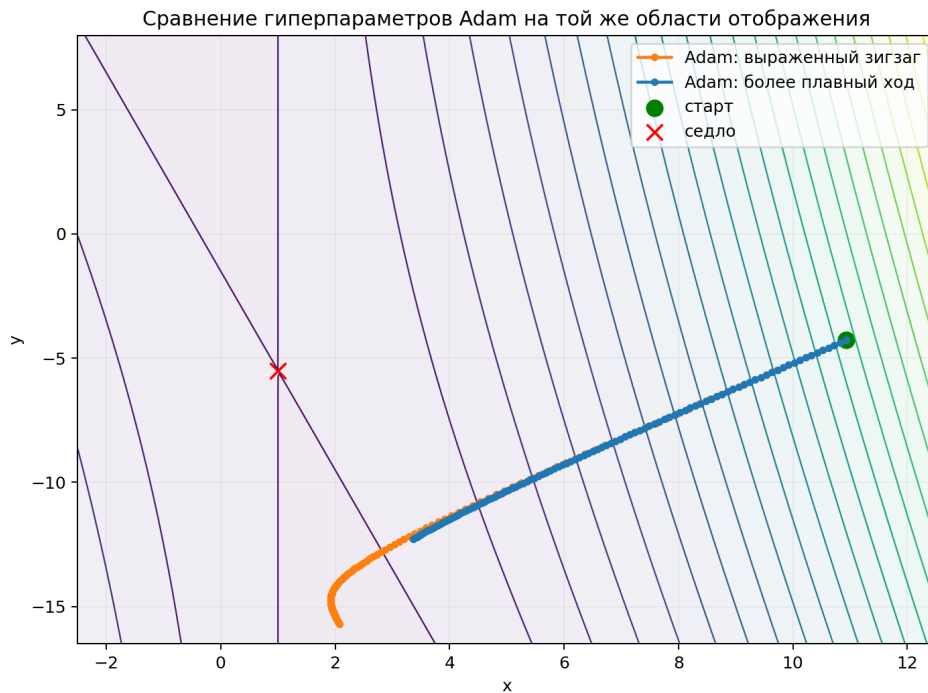


Рис. 3: Сравнение гиперпараметров Adam на той же области отображения

3.5 Готовая реализация Adam из PyTorch

Для проверки использован `torch.optim.Adam` с теми же гиперпараметрами, что и у собственной реализации.

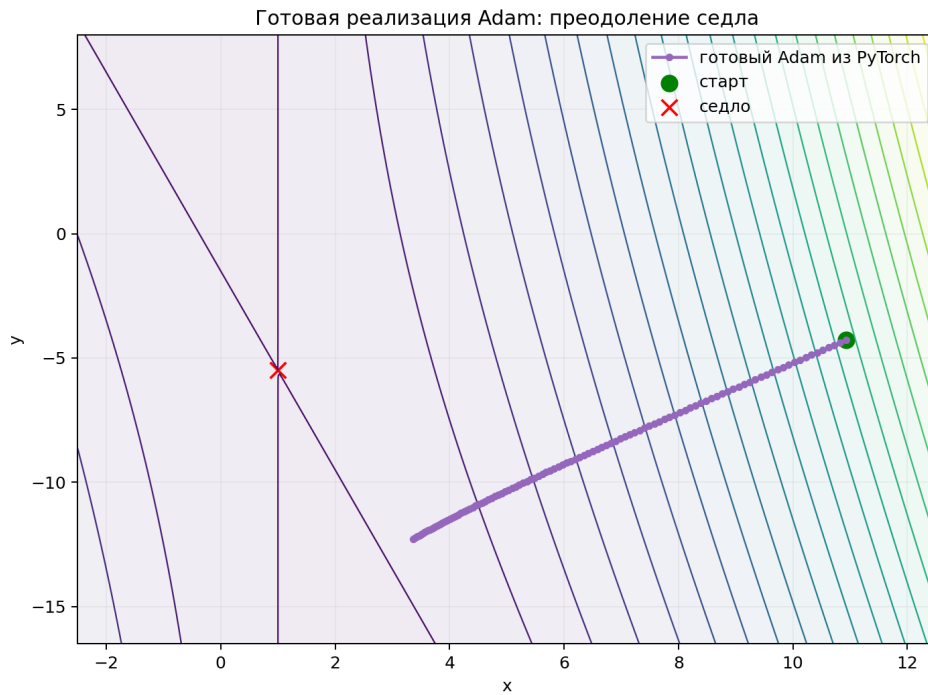


Рис. 4: Готовая реализация Adam: преодоление седла

3.6 Вывод по заданию 2

- обычный градиентный спуск на выбранном старте действительно сходится к седловой точке;
- Adam уводит траекторию от седла и позволяет продолжить движение вниз по функции;
- собственная реализация Adam даёт практически ту же динамику, что и встроенная реализация PyTorch;
- различия в траекториях и метриках объясняются только численным шумом и особенностями внутренней реализации.

4 Задание 3. Нейросеть на табличных данных

4.1 Выбор датасета

Для задания выбран датасет **UCI Bank Marketing**:

- источник: UCI Machine Learning Repository;
- дата публикации: **2012**;
- размер: **41 188** объектов;
- число исходных признаков: **20**;
- тип задачи: бинарная классификация.

После предобработки:

- числовые признаки стандартизированы;
- категориальные признаки закодированы one-hot кодированием;
- итоговая размерность входа: **63**.

4.2 Архитектура модели

Использована MLP-сеть:

$$63 \rightarrow 64 \rightarrow 32 \rightarrow 1 \quad (13)$$

Слои:

- **Linear** — линейное преобразование;
- **ReLU** — нелинейность;
- **Dropout(0.1)** — регуляризация;
- и так далее.

Функция потерь: `BCEWithLogitsLoss` с весом для положительного класса.

4.3 Сравнение оптимизаторов

Сравнивались:

- стандартный `torch.optim.Adam`;
- собственный `AdamVariant`.

4.3.1 Результаты

Таблица 1: Сравнение метрик оптимизаторов на тестовом наборе

Оптимизатор	Асс	F1	ROC AUC	Val loss	Эпохи	Время, с
PyTorch Adam	0.8399	0.5719	0.9499	0.4950	20	6.51
Собственный Adam	0.8394	0.5717	0.9499	0.4956	20	7.62

4.3.2 Матрицы ошибок

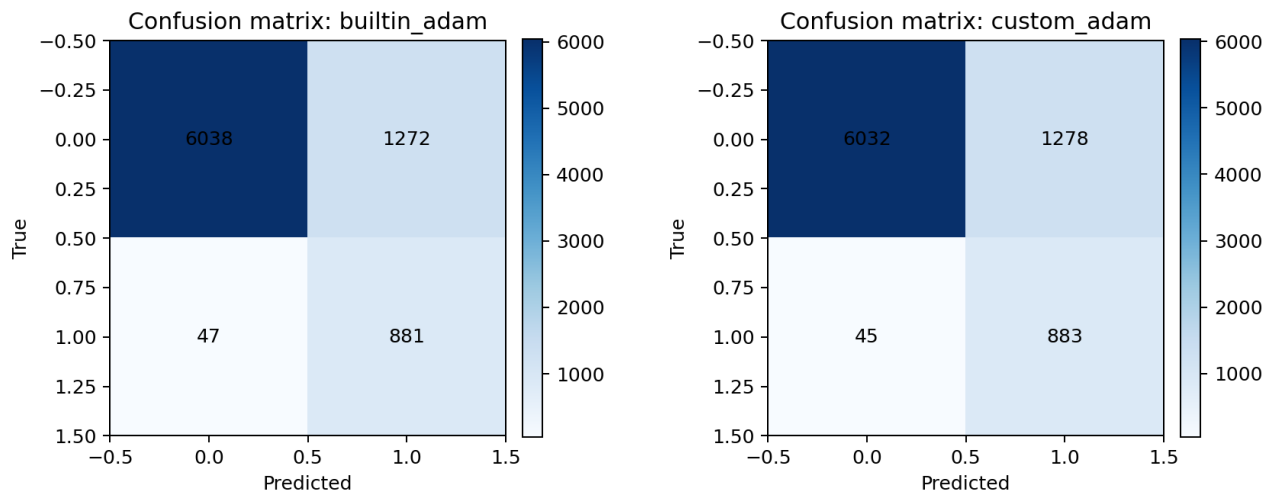


Рис. 5: Матрицы ошибок: слева — PyTorch Adam, справа — собственный Adam

4.3.3 Кривые обучения

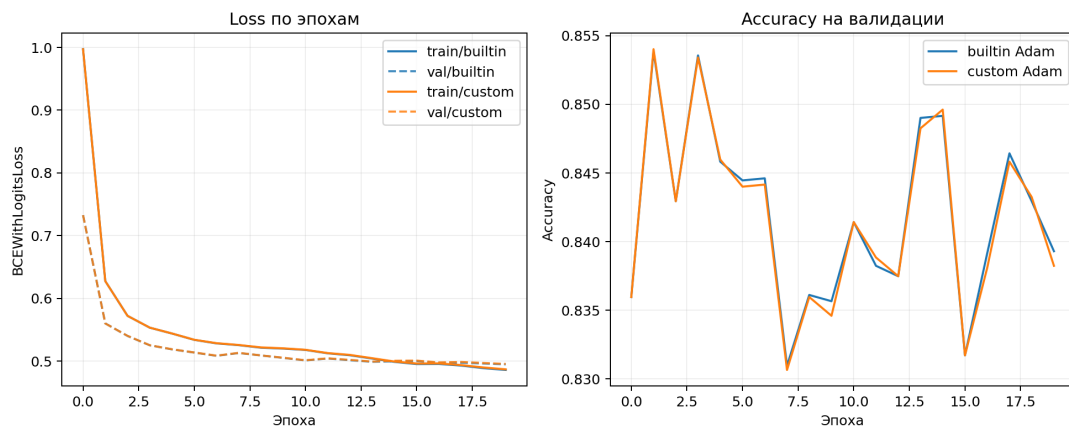


Рис. 6: Кривые обучения по эпохам

4.4 Вывод по заданию 3

Собственная реализация оптимизатора повторяет поведение встроенного Adam с очень близкими метриками качества.

Небольшое различие по времени обучения ожидаемо: собственный оптимизатор реализован на Python и работает медленнее, чем оптимизатор из библиотеки.

5 Итоговый вывод

В работе:

1. найдено седло функции и исследованы экстремумы;
2. построены линии уровня;

3. показано застревание обычного градиентного спуска около седла;
4. реализован собственный Adam и проверен на той же задаче;
5. выполнено сравнение с готовым Adam из PyTorch;
6. обучена нейросеть на табличном датасете, получено сопоставимое качество для встроенного и собственного оптимизатора.